

Ayaan Khan | DS-D | 22I-2066

Deep Learning for Perception

Assignment #1

Network details, training settings & rationale:

Architectures used:

- **ResNet-50** (pretrained on ImageNet) — 25.1M parameters. Good trade-off: robust, widely used, easy to fine-tune.
- **EfficientNet-B3** (pretrained) — 11.9M parameters. Modern, efficient, strong accuracy per FLOP.
- **VGG-16** (pretrained) — 137.4M parameters. Classic deep CNN; included to show effect of older large models vs modern architectures.
- **MobileNet-V2** (pretrained) — 3.2M parameters. Lightweight, mobile friendly baseline.

Model heads (same multi-task head design):

- Shared backbone → two heads:
 - Classification head: Linear(feature_dim → 512) → BN → ReLU → Dropout(0.5) → Linear(512 → 8), CrossEntropyLoss (label_smoothing=0.1).
 - Regression head: Linear(feature_dim → 256) → ReLU → Dropout(0.5) → Linear(256 → 2), MSELoss for valence & arousal.

Training settings (what was used for experiments):

- Input size: 224×224 .
- Augmentation (on-the-fly): RandomResizedCrop, RandomHorizontalFlip, ColorJitter, RandomRotation.
- Optimizer: AdamW (default experiments used lr tuned per model; in improved experiments lr=3e-4 for many runs, but per-model overrides were tested).
- Scheduler: CosineAnnealingLR (T_{max} = epochs).

- Mixed precision: AMP (GradScaler + autocast).
- Batch size: 32 (reduce to 16 if OOM). num_workers=0 for Windows.
- Multi-task loss: $\text{loss} = \text{CE} + \lambda * \text{MSE}$ ($\lambda = 0.5$).

Rationale for baselines:

ResNet50 and EfficientNet (modern, strong) were selected as primary baselines; VGG16 and MobileNetV2 added to represent a heavy old model and an efficient mobile model for contrast. Transfer learning (ImageNet pretrained weights) was used for quicker and more reliable convergence on a relatively small dataset (~4k images).

Comparison of baselines

Training times (measured wall-clock)

- **50 epochs:**
 - ResNet-50 — *18 m 40 s*
 - EfficientNet-B3 — *20 m 50 s*
 - VGG-16 — *30 m 58 s*
 - MobileNet-V2 — *18 m 58 s*
- **25 epochs:**
 - ResNet-50 — *12 m 22 s*
 - EfficientNet-B3 — *12 m 19 s*
 - VGG-16 — *17 m 23 s*
 - MobileNet-V2 — *11 m 14 s*

Quantitative results (validation set — table produced from test runs)

Model	Num of params	acc	f1_macro	kappa	kripp_alpha	auc_roc_ovr	auc_pr_macro
ResNet50	25,087,306	0.4950	0.4919	0.4229	0.4224	0.8430	0.5084
EfficientNet-B3	11,882,290	0.4725	0.4701	0.3971	0.3962	0.8127	0.4540
VGG16	137,412,682	0.3725	0.3517	0.2829	0.2790	0.8077	0.3827

MobileNet-V2	3,213,322	0.4800	0.4737	0.4057	0.4037	0.8461	0.5031
---------------------	-----------	--------	--------	--------	--------	--------	--------

Model	val_rmse	aro_rmse	val_corr	aro_corr	val_sagr	aro_sagr	val_ccc
ResNet50	0.3714	0.3181	0.6113	0.5349	0.7713	0.8150	0.5445
EfficientNet-B3	0.3966	0.3367	0.5608	0.4822	0.7550	0.8063	0.5312
VGG16	0.4133	0.3430	0.4820	0.4015	0.7250	0.7863	0.3310
MobileNet-V2	0.3859	0.3266	0.5855	0.5260	0.7438	0.8125	0.5553

Brief comparison & takeaway:

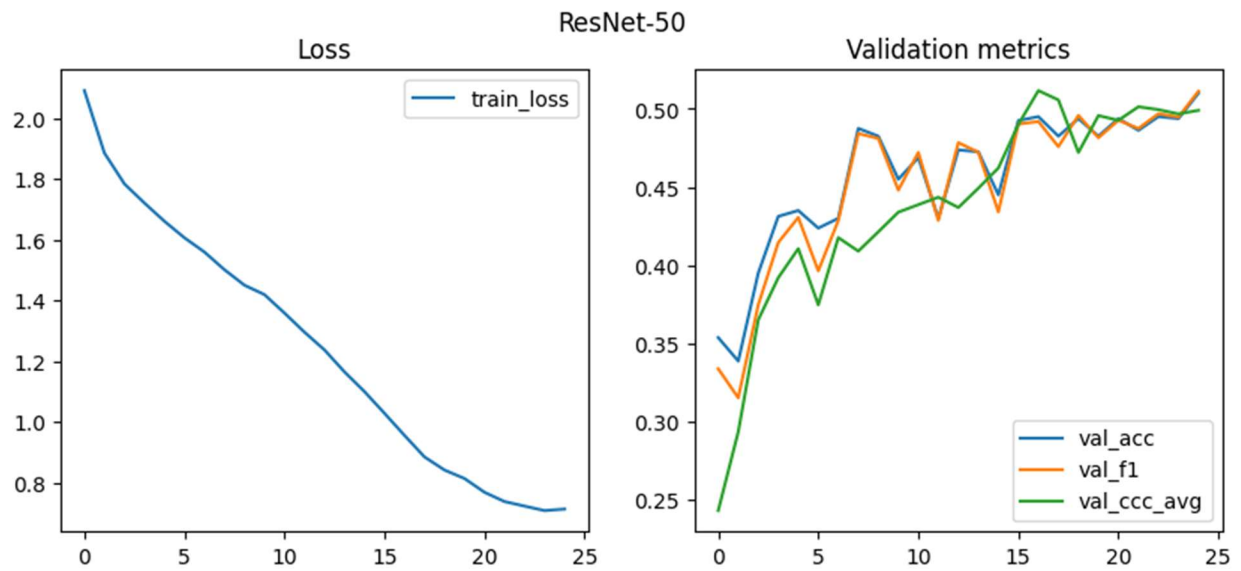
- **ResNet-50** achieved the highest accuracy (0.495) and highest F1 among these runs — best overall trade-off of accuracy and regression CCC.
- **EfficientNet-B3** performs close behind ResNet in some metrics and is far smaller; good FLOP/accuracy trade-off.
- **MobileNet-V2** surprisingly performed competitively given tiny parameter count.
- **VGG-16** performed worst on this dataset, likely due to heavy overparameterization and overfitting with limited data.

3. Transfer learning details:

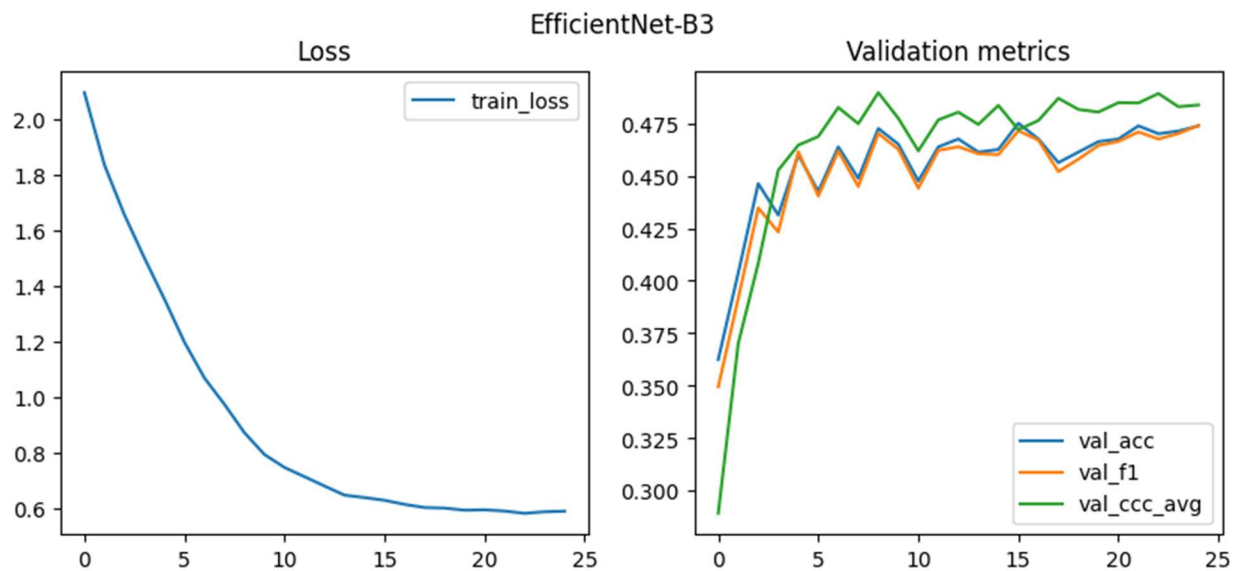
- All backbones were initialized with ImageNet weights (PyTorch weights="IMAGENET1K_V1").
- Initially we used *head-only* training (freeze most backbone layers) for quick warm-up experiments, then fine-tuned entire network (unfreeze) for final runs.
- Checkpoints saved as best_<backbone>.pth (saved best by average CCC across valence & arousal).

4. Training graphs

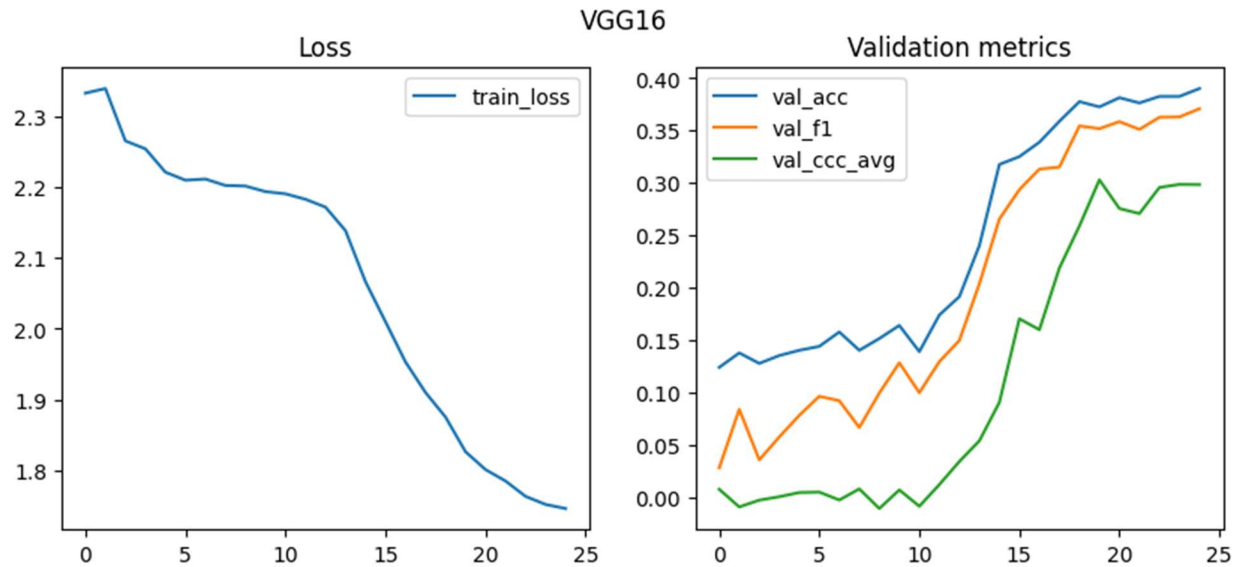
ResNet-50:



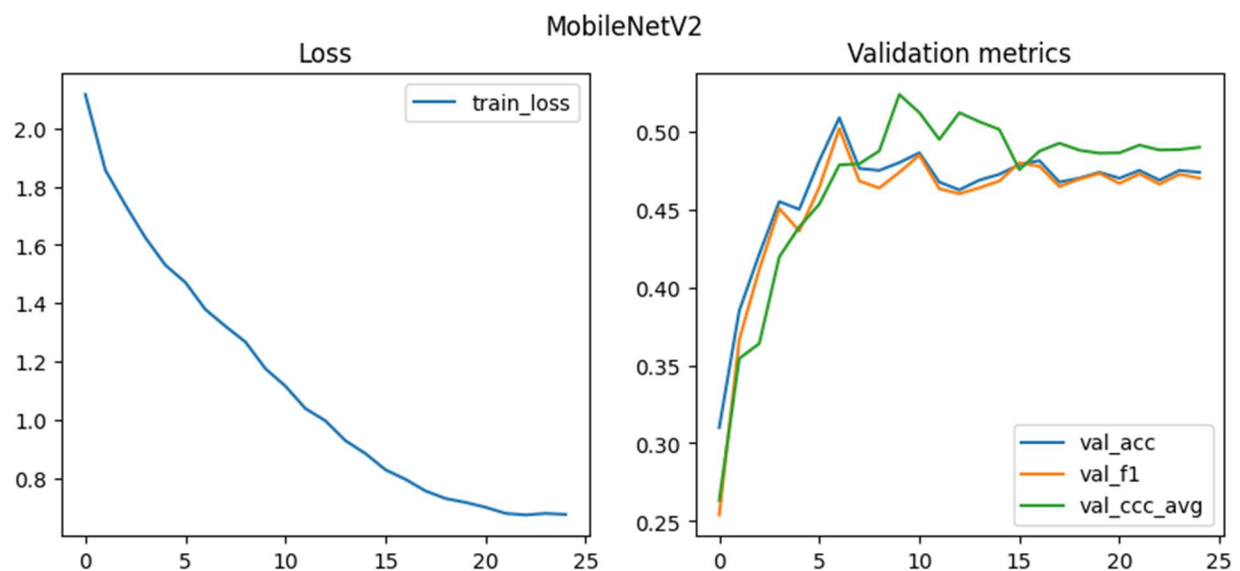
EfficientNet-B3:



VGG-16:



MobileNet-V2:



5. Continuous-metrics: RMSE, CORR, SAGR, CCC — rationale & recommendation

Definitions & intuition

- RMSE (Root Mean Square Error)** — measures absolute difference between predicted and true continuous values (valence/arousal). Lower is better. Useful to quantify average magnitude of prediction error.

- **CORR (Pearson correlation)** — measures linear association between predictions and ground truth (ignores scale differences). High correlation indicates the model captures trends/variation well.
- **SAGR (Sign Agreement)** — fraction where predicted and true values share the same sign (positive/negative). Important in affect because sign indicates overall positive/negative affect — getting the sign right is often more important than exact magnitude.
- **CCC (Concordance Correlation Coefficient)** — combines Pearson correlation with mean offsets to penalize both disagreements in scale and in location; recommended as a holistic metric for continuous affect prediction.

Which metric matters most for a system that should work “in the wild”?

- **CCC** is the best single metric because it captures both correlation and bias (scale/mean differences).
- **SAGR** is also important operationally when deciding positive vs negative affect is the primary application (e.g., sentiment flagging).
- **RMSE** is useful when downstream systems rely on absolute valence values (e.g., thresholds).
- **Recommendation:** prioritize **CCC** and **SAGR** for field-robust performance; report RMSE and Pearson as supporting statistics.

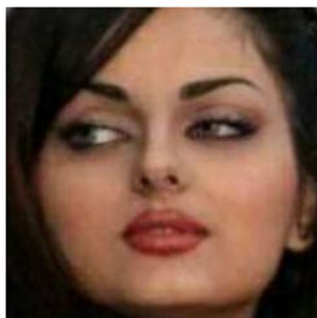
6. Qualitative results

ResNet-50:

Correct:

ResNet50 — Correct examples (3)

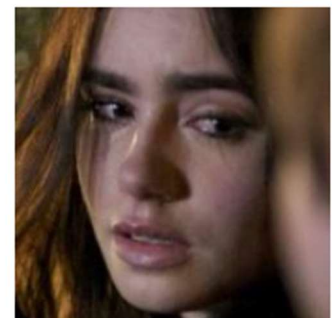
GT:5
Pred:5
P:0.30
Val G/P:-0.87/-0.30
Aro G/P:0.44/0.38



GT:3
Pred:3
P:0.97
Val G/P:0.31/0.09
Aro G/P:0.68/0.73



GT:2
Pred:2
P:0.54
Val G/P:-0.68/-0.41
Aro G/P:-0.39/0.16



Incorrect:

ResNet50 — Incorrect examples (3)

GT:7
Pred:2
P:0.85
Val G/P:-0.64/-0.51
Aro G/P:0.64/-0.11



GT:2
Pred:1
P:0.40
Val G/P:-0.46/0.23
Aro G/P:-0.21/0.31



GT:4
Pred:6
P:0.59
Val G/P:-0.15/-0.25
Aro G/P:0.79/0.52



EfficientNet-B3:

Correct:

EfficientNet-B3 — Correct examples (3)

GT:7
Pred:7
P:0.37
Val G/P:-0.64/-0.23
Aro G/P:0.64/0.30



GT:3
Pred:3
P:0.94
Val G/P:0.31/0.17
Aro G/P:0.68/0.89



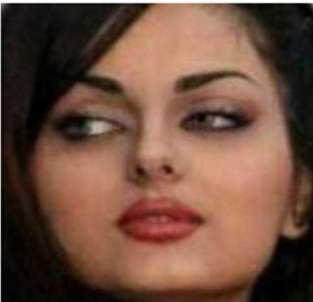
GT:3
Pred:3
P:0.98
Val G/P:0.33/0.15
Aro G/P:0.83/0.82



Incorrect:

EfficientNet-B3 — Incorrect examples (3)

GT:5
Pred:0
P:0.90
Val G/P:-0.87/-0.06
Aro G/P:0.44/-0.00



GT:2
Pred:1
P:0.64
Val G/P:-0.46/0.32
Aro G/P:-0.21/0.32



GT:2
Pred:7
P:0.74
Val G/P:-0.68/-0.50
Aro G/P:-0.39/0.31



VGG-16:

Correct:

VGG16 — Correct examples (3)

GT:7
Pred:7
P:0.34
Val G/P:-0.64/-0.27
Aro G/P:0.64/0.27



GT:3
Pred:3
P:0.34
Val G/P:0.31/-0.11
Aro G/P:0.68/0.53



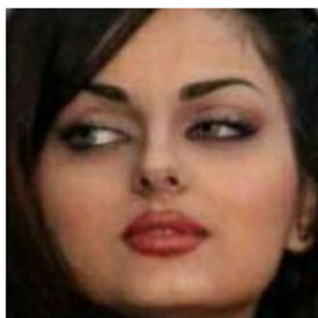
GT:3
Pred:3
P:0.63
Val G/P:0.33/-0.01
Aro G/P:0.83/0.79



Incorrect:

VGG16 — Incorrect examples (3)

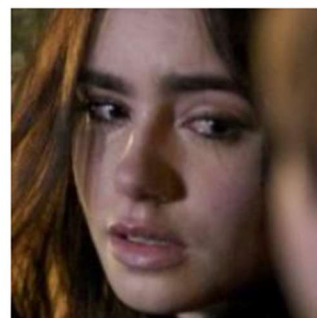
GT:5
Pred:0
P:0.31
Val G/P:-0.87/-0.23
Aro G/P:0.44/0.25



GT:2
Pred:1
P:0.63
Val G/P:-0.46/0.16
Aro G/P:-0.21/0.21



GT:2
Pred:0
P:0.27
Val G/P:-0.68/-0.22
Aro G/P:-0.39/0.25



MobileNetV2:

Correct:

MobileNet-V2 — Correct examples (3)

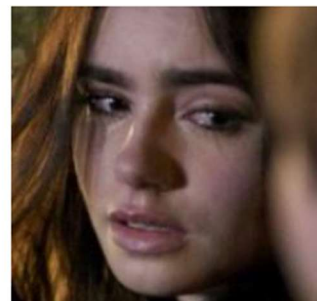
GT:7
Pred:7
P:0.31
Val G/P:-0.64/-0.47
Aro G/P:0.64/0.28



GT:3
Pred:3
P:0.98
Val G/P:0.31/0.20
Aro G/P:0.68/0.99

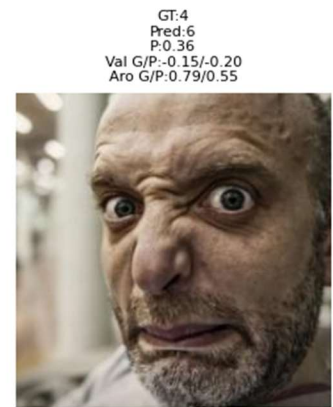


GT:2
Pred:2
P:0.38
Val G/P:-0.68/-0.43
Aro G/P:-0.39/0.14



Incorrect:

MobileNet-V2 — Incorrect examples (3)



Short discussion, limitations & future work

- **Why ResNet/EfficientNet did best:** modern architectures capture more discriminative features and generalize better when fine-tuned with augmentations.
- **Why VGG performed poorly:** very large number of parameters → overfitting on modest dataset size; regularization and more aggressive augmentation or dropout might help, but switching to modern backbones is more effective.
- **Possible improvements:**
 - Balanced class sampling or class reweighting if expression classes are imbalanced.
 - Semi-supervised learning or self-supervised pretraining (SimCLR, BYOL) if unlabeled data is available.
 - Mixup / CutMix, or offline augmentation to expand dataset.
 - Ensemble of top backbones (ResNet + EfficientNet) for improved robustness.
 - Test calibration methods for classification probabilities.

Final recommendation:

Use **ResNet-50** or **EfficientNet-B3** as primary models (ResNet gave slightly higher accuracy in these runs), but prefer **EfficientNet-B3** for deployments where compute/efficiency trade-off matters—report CCC and SAGR as primary continuous metrics for field readiness.

Link to GitHub Repository:

<https://github.com/AyaanKhan0111/FacialAffectPyTorchCNN>